



TIMELYLLM: Time-sensitive LLM Serving System for Physical-I/O Limited Agents

Neiwen Ling

Yale University

New Haven, Connecticut, USA

neiwen.ling@yale.edu

Anurag Khandelwal

Yale University

New Haven, Connecticut, USA

anurag.khandelwal@yale.edu

Guojun Chen

Yale University

New Haven, Connecticut, USA

guojun.chen@yale.edu

Lin Zhong

Yale University

New Haven, Connecticut, USA

lin.zhong@yale.edu

Abstract

Large Language Models (LLMs) are increasingly integrated into Physical-I/O limited agents, such as robots and voice assistants, which execute outputs sequentially. However, existing LLM serving systems typically employ a throughput-oriented batching mechanism, ignoring the large gap between LLM generation speed and the constrained physical I/O rates of agents, thus wasting execution slack and worsening resource contention. Besides, they treat all tokens equally and cannot anticipate the execution implications of different content, preventing scheduling aligned with agent-side behavior. To address it, we propose a new system named TIMELYLLM that coordinates LLM generation with the physical behavior of agents. TIMELYLLM introduces a novel segmented generation and scheduling mechanism, strategically leveraging the time gap between agent plan generation and execution to reduce contention and improve response latency under multi-agent workloads. We implement TIMELYLLM on top of a widely-used LLM serving framework. We also build a dataset collection system to construct serving workloads from real-world robots, including drones, robot arms, and quadruped robots. Our evaluation demonstrates that TIMELYLLM improves the time utility up to 1.52 $\times$, and reduces the overall waiting time by 84%.

CCS Concepts

• **Computer systems organization** $\rightarrow$ **Real-time system architecture**; *Robotics*.

Keywords

large language models, multi-agent systems, time-sensitive serving, robotic agents, slack-based scheduling, segmented generation

1 Introduction

Large Language Models (LLMs) are increasingly used as the reasoning engine for both physical agents [11, 42, 43, 52, 62], such as drones, robot arms, and quadruped robots, and virtual agents

such as voice assistants [7, 20, 22, 57]. As these agents interact with the physical world, including human users and other agents, such interactions pose both a challenge and an opportunity for the LLM serving: they can spend a considerable amount of time waiting for the physical I/O; on the other hand, their reasoning and action are often time-sensitive, even deadline-bound, for responsiveness and reliability.

First, waiting for the physical I/O creates opportunities to make the LLM serving more efficient. Agents' interactions with the world are limited in speed by both physics and human nature, often orders of magnitude slower than LLM token generation. For instance, GPT-4o [21] can generate close to 100 tokens per second, while a drone may take a few seconds to execute a simple command of a few tokens, like moving forward, and a voice agent usually speaks fewer than 200 words per minute. Stream interpretation in TypeFly [11, 12] allows a physical I/O action to start as soon as it is detected in the LLM output so that the agent can act even before the complete plan is generated. As the physical I/O action can take a long time, there is indeed no need to continue generating the plan; instead, the serving system could reallocate computational resources to serve more urgent requests. Existing LLM serving engines are not equipped to leverage this opportunity. The LLM output tokens are transparent to them. They cannot tell whether a sequence of tokens encodes a long-horizon robot movement, a short condition check, or a complete conversational sentence.

On the other hand, agents are often time-sensitive and even deadline-bound in their interactions with the physical world. As a result, exploiting execution slack requires fully runtime-adaptive scheduling, in which the LLM serving system must decide when a request should yield computation, when it should resume generation, and how resources should be redistributed across concurrently executing agents. However, current LLM serving engines are not designed to support such dynamic control. Existing engines (e.g., vLLM and TensorRT-LLM) [6, 26, 35, 59] typically rely on continuous batching with simple policies such as first-come-first-served (FCFS) or static priority [8, 38, 44]. Under these schemes, the generation process for a request is non-preemptible and remains in the batch until completion. Physical I/O activities can also be viewed as a special case of tool use. LLM tool use usually happens after the complete response is generated by the LLM. Recent LLM serving work allows tool use to start before generation completion [18]. In neither case, the serving system exploits the generation-execution



This work is licensed under a Creative Commons Attribution 4.0 International License. *MobiSys '26, Cambridge, United Kingdom*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2027-7/26/06

<https://doi.org/10.1145/3745756.3809203>

Table 1: LLM Latency for a Single Request Across Different Phases: Tokenization, Decoding, and Detokenization. The table demonstrates that decoding dominates the entire inference procedure. We test Llama3-8B-float16 and Phi-3-mini-128k-float16, implementing them using Hugging Face Transformers [15] on an NVIDIA RTX 4090.

Phase	Tokens	Llama3	Phi3
Tokenization	2884	2.609 ms (0.43%)	1.541 ms (0.18%)
Decoding 0	token 0	328.45 ms	563.04 ms
Decoding 1	token 1	21.93 ms	22.24 ms
Decoding 2	token 2	21.56 ms	20.99 ms
3 to N-1	...	...	...
Decoding N	last token	21.77 ms	22.00 ms
Decoding	15	611.2 ms (99.57%)	841.7 ms (99.80%)
Detokenization	15	0.052 ms (0.008%)	0.091 ms (0.011%)
Total time	in:2884, out:15	613.82 ms	843.36 ms

gap of Physical-I/O limited agents. More broadly, existing systems primarily optimize the tradeoff between throughput and latency, without explicitly accounting for the time sensitivity of requests.

In response to the opportunity and challenges discussed in the above, we develop TIMELYLLM, a time-sensitive LLM serving system for concurrent requests from Physical-I/O limited agents. TIMELYLLM is based on two complementary ideas: (i) First, it analyzes the LLM output simultaneously with the generation and can suspend the generation process, breaking it into segments based on the output content. (ii) Second, TIMELYLLM employs runtime-adaptive scheduling to manage the segmented generation processes, ensuring compliance with the time-sensitive demands of each request. For example, in robotic task planning, the system generates the initial segment within the tolerable response time (e.g., 1 s to maintain smooth command-driven human-computer interaction [47]) and subsequent segments before completing the preceding plan execution. In this way, the system effectively utilizes the time gap between the response generation and agent execution to disperse the workload within a time duration, thereby alleviating resource contention and maximizing compliance with the time-sensitive demands of each request.

We implement TIMELYLLM on top of the widely-used LLM serving framework vLLM [26] and deploy it on two real-world case studies. We also build a dataset collection system LRTrace to construct serving workloads from real-world robots, including drones [50], robot arms [34], and quadruped robots [51], aiding in the evaluation of multi-agent LLM serving systems. Experimental results show that compared to the state-of-the-art vLLM, TIMELYLLM achieves up to a 1.52 $\times$ improvement in the time utility of robotic task response time, and reduces the overall robot waiting time by 84%. The source code of TIMELYLLM and our dataset collection system LRTrace is openly available at [33]. Detailed instructions for reproducing our core experiments are provided in Appendix C.

We summarize the contributions of this paper as follows:

- We present TIMELYLLM, the first time-sensitive LLM serving system for physical I/O limited agents, which better meets the time requirements of concurrent requests by leveraging the time gap between LLM plan generation and agent execution.
- We describe an efficient segmented generation mechanism with a content-aware stop checker and context caches. We also design a segmented scheduling strategy considering both time utility gain

Table 2: Consumption rates of I/O limited agents.

Robot Type	Forward	Backward	Left	Right	Turn
Drone(Ryze Tello)	2.53s	2.53s	2.53s	2.53s	1.53s
Quadruped (Unitree Go2)	1.50s	1.50s	2.47s	2.47s	0.80s
Self-deployed Robot Car	3.87s	3.87s	4.08s	4.08s	1.17s

(a) Robot-side action execution time ($d = 1$ m, $\theta = 45^\circ$).

	ChatGPT	Gemini	Meta AI	Siri
Time(20 words)	8.43s	7.7s	8.14s	7.58s
Avg Words Per Minute	142	156	147	158

(b) Speech playback speed of major spoken agents.

and agent execution efficiency, enabling our system to handle multiple requests with optimized time utility.

- We design and build LRTrace, the first dataset collection system that enables the flexible creation of real-world time traces for multiple LLM-powered robots.

2 Background

Latency Profile of LLM Service. A typical LLM service processes each incoming request in three stages: tokenization, decoding, and detokenization [9, 31]. *Tokenization* converts text into a sequence of tokens that the model can process. *Decoding* predicts each subsequent token based on the tokens generated so far. Finally, *Detokenization* converts the output tokens back into human-readable text. Decoding is the most time-consuming stage. Table 1 presents the latency measurement for serving one request with both Meta Llama3 [31] and Microsoft Phi-3 [4]. In both cases, the decoding phase accounts for over 99% of the total time. Because the model generates tokens sequentially, the latency grows linearly with respect to the number of output tokens [11]. Decoding the first token (token 0 in Table 1) is particularly expensive because it requires *Prefilling* that computes the attention states for input tokens. This computation is unnecessary for subsequent tokens due to the use of the KV cache [39] and for shared prompt segments, thanks to Prompt cache [17].

To improve GPU utilization and system throughput, LLM serving engines such as vLLM [3] and TensorRT-LLM [35] commonly adopt batch processing. We build TIMELYLLM on top of *continuous batching* [26, 59], which allows a new request to be added for processing as soon as resources become available. Batching, including continuous batching, trades the service latency of individual requests for high resource utilization. Moreover, most batching systems [6, 26, 35, 59] employ the First-Come, First-Served (FCFS) scheduling policy that ignores the urgency of requests. To tackle these challenges, our system TIMELYLLM introduces a novel LLM scheduling mechanism that strategically prioritizes and batches requests based on their time-sensitive requirements.

Physical I/O limited Agents and Latency Model. In LLM-powered robot control, robots leverage the LLM’s reasoning capabilities for planning a task given by the user instructions [11, 52, 61] or handling unexpected situations [29, 48, 49]. Due to the use of in-context learning, robotic requests to the LLM are typically very long, encompassing the user’s instructions, information about the world,

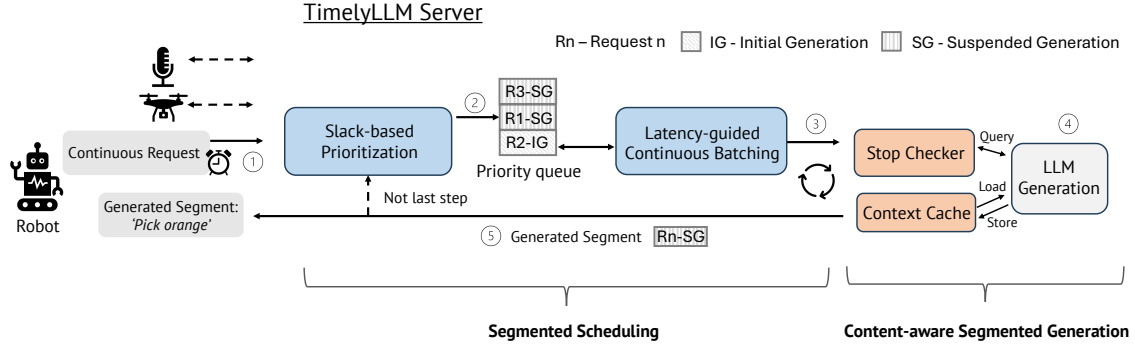


Figure 1: System overview of TIMELYLLM: A agent submits a request to TIMELYLLM. TIMELYLLM generates the response for incoming requests in segments by continuously suspending and resuming the generation (§4). The scheduler in TIMELYLLM (§5.1) manages all the initial and suspended generation and (§5.2) prioritizes them based on the slack time window. Additionally, TIMELYLLM adaptively adjusts batch sizes to efficiently harness GPU parallel computing resources (§5.3).

the robot’s skill sets, guidance for plan generation, and case examples. The output is typically a sequence of robot actions, e.g., ‘pick(cyan_box)->place(cyan_box)-> pick(red_box)’ for a robot arm [52].

Spoken agents form another major category of physical-I/O limited LLM applications. Modern assistants, such as Meta’s AI assistant [32], allow users to ask open-ended questions and receive natural-language responses generated by LLMs. These systems convert each generated token into audio and play it back incrementally. Unlike text chat, where users visually scan at their own pace, speech playback is constrained by human auditory perception: natural speech is typically processed at roughly 150 WPM (Table 2). This human hearing rate effectively fixes the allowable playback speed.

Despite their distinct physical behaviors, both agents share a critical characteristic: they consume LLM outputs sequentially at a rate significantly slower than generation, thereby imposing a fundamental Physical-I/O limitation. In both scenarios, two latencies of the LLM service are important to the end user [11]. The first, called *response time*, is the time from when the service receives a task to when the agent begins to act or speak. The second, which we refer to more generally as *agent-side completion time*, measures the delay until the agent finishes executing the planned actions or finishes playing the generated audio. This metric comprises two parts: agent waiting time, defined as the total latency introduced by the LLM generation, and agent execution time, which depends on the content of the generated sequence.

3 Overview of TIMELYLLM

TIMELYLLM aims to provide time-sensitive LLM services for concurrent and continuous queries from multiple physical-I/O limited agents. In such shared-resource settings, the system must ensure that each agent receives the response in time, despite many requests contending for the same compute backend.

To address this challenge, TIMELYLLM leverages the agent’s execution time as natural slack: LLM does not need to generate the full response immediately, but only needs to finish generation before

the agent exhausts the available content. This slack allows TIMELYLLM to interleave segment generation across concurrent requests, substantially reducing contention on the shared LLM backend. As shown in Figure 1, upon receiving a request, TIMELYLLM seeks to emit an initial, executable segment as early as possible so that the agent can begin acting or speaking promptly. After delivering this first piece of content, TIMELYLLM suspends decoding and returns the request to the scheduler, allowing compute resources to be allocated to other, more urgent requests. TIMELYLLM resumes decoding only when the scheduler determines that it is the appropriate time to continue generation. This design prevents overgeneration and enables the LLM to reallocate compute resources to other agents while the current one is executing or playing back.

To realize it, TIMELYLLM performs content-aware segmented generation. During decoding, the system continuously monitors the output stream and identifies self-contained, executable segments, such as a complete robot action or a coherent clause for speech output. When the system detects such a segment, it immediately suspends decoding and sends the segment to the agent. This mechanism ensures early response while providing natural suspension points for subsequent scheduling decisions (§4).

Coordinating many concurrent initial and suspended generations requires a timing-aware scheduling strategy. TIMELYLLM frames this as a slack-based prioritization problem: each segment has a finite slack window determined by how long the agent will take to consume the previous segment. If the system completes generation within this window, the agent proceeds without user-perceived delay. TIMELYLLM continuously tracks slack across all active requests and prioritizes the tasks with the least remaining slack, aligning decoding progress with agent-side consumption (§5.2).

Finally, TIMELYLLM integrates latency-guided continuous batching. While batching improves GPU efficiency, it increases per-step decoding latency and may violate the slack constraints of time-critical segments. Before expanding the batch, the system performs a lightweight feasibility check to ensure that the most urgent requests can still meet their slack deadlines (§5.3).

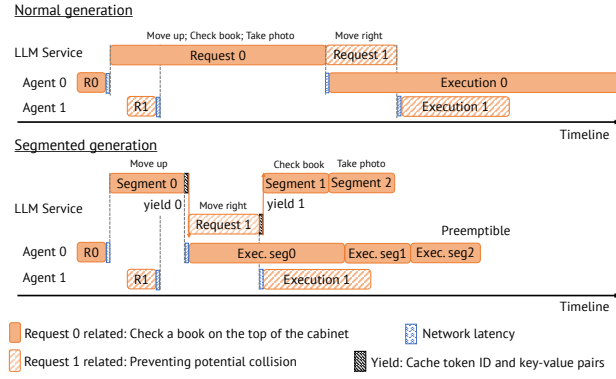


Figure 2: Normal vs. Content-aware Segmented Generation on a serving system capable of serving one request at a time (batch size = 1): With normal generation (top), the system will not serve Request 1 until it finishes serving Request 0. With Content-aware segmented generation (bottom), the system can switch to serve Request 1 after it generates Segment 0 of Request 0. Once Request 1 is completed, the system resumes Request 0 and generates Request0-Segment1, which completes before the execution of Request0-Segment0 finishes. There is no network latency shown before the execution of Segment 1, as the plan transmission is completed during the execution of Segment 0.

4 Content-aware Segmented Generation

We first present our content-aware segmented LLM generation. The keys are two: (i) identifying the right time for segmentation, and (ii) context switching, both of which must be done efficiently.

Segmentation for agent plans. To generalize TIMELYLLM without coupling the serving engine to specific agent logic, we abstract the segmentation module as a pluggable *SegmentChecker* interface. Within the decoding loop, this interface validates feasibility based on: (i) Structural Completeness, ensuring the token sequence forms a valid executable unit (e.g., action primitive, or speech clause); and (ii) Time Compliance, verifying that the estimated execution or playback time exceeds a threshold. This constraint prevents scheduling inefficiencies caused by trivial micro-segments (e.g., short interjections). Importantly, segmentation is triggered only at complete executable units. Intermediate reasoning tokens or partial structures do not trigger suspension and are treated as part of the same logical block. Beyond our prototype, the interface is compatible with programmable serving systems [19], which allows users to register sandboxed “inferlets” for custom stop logic, and supports standard instruction-following protocols via system prompts.

In this work, we instantiate this interface using compiled heuristics to minimize runtime latency. TIMELYLLM employs incremental detokenization to detect specific robotic skills (e.g., ‘pick’ in [52], ‘mf’ in [11]) or linguistic boundaries in real-time. As shown in Figure 2, generation is suspended immediately upon validation. As demonstrated in Table 1, the incremental detokenization accounts for only about 0.01% of the total generation time, as TIMELYLLM shifts the detokenization process into the generation loop without performing redundant computations.

Context switching. When suspending a generation, TIMELYLLM retains its KV cache as the intermediate state as well as the tokenized output so far, which constitutes the context of the generation. When resuming the generation, TIMELYLLM only needs to restore its context. We chose to use tokenized output as part of the context for two reasons. First, it obviates retokenization upon resumption, reducing latency, especially for long prompts. Second, it prevents mismatches that can occur during retokenization. For instance, encoding the token IDs “[30, 344, 493, 2239, 873, 419, 2575]” produces the text “?iv(‘book’)==True”, but retokenizing this text might yield a different token sequence like “[30, 344, 493, 2239, 99991, 2575]”. The memory overhead for storing the tokenized output is reasonably modest, requiring only 144 bytes to store 13 tokens for a concise robot plan.

Saving the context of a suspended generation adds pressure to the memory hierarchy of TIMELYLLM. As active generations are likely to exhaust the rather limited GPU High-bandwidth Memory (HBM), the context of a suspended generation is likely to be evicted by the underlying memory management subsystem from GPU HBM and reside in the more abundant host memory. This leads to overhead when the suspended generation is resumed: TIMELYLLM must retrieve its context from the host memory to the GPU HBM. However, for lengthy robotic requests, storing the KV cache still remains more efficient than re-prefilling. For instance, the context for generating single-robot control using Llama3-8B-float16 includes 170.35 MB of the KV cache and 0.012 MB of tokenized output. Transferring this data from host memory to GPU HBM takes 9.50 ms, less than 3% of the total generation time (~ 370 ms), whereas re-prefilling the inputs takes significantly longer at 133.31 ms. Moreover, with pipelined generation and execution, context restoration often overlaps with execution, reducing the perceived overhead. As shown in our evaluation (§7.4), segmented generation improves response time and time utility, even under high concurrency, indicating that its benefits outweigh the incurred cost in practical scenarios.

5 Segmented Scheduling

5.1 Problem Formulation

TIMELYLLM features a segmented LLM scheduling mechanism designed to control the generations with the goal of maximizing the aggregate time utility of concurrent agent requests. We next introduce the task model and objective function of our system.

LLM request model. We define the set of LLM requests as the collection of requests received from agents, denoted by $\mathbb{T}$. Requests can arrive concurrently and continuously. We represent the i th received request as r^i , where i from 0 to I , and quantify its mission criticality using a Time Utility Function (TUF). While our scheduling algorithm applies to any monotonically non-increasing utility function, we instantiate the TUF in this work using a standard linear decay model [30]. Specifically, we characterize the mission criticality by the tuple $(\beta^i, \alpha^i, ERT^i)$ and define the utility as $U(t) = \min(\beta, \alpha(t - ERT) + \beta)$. Here, β^i (Base Importance) denotes the maximum utility before the Expected Response Time (ERT^i), beyond which utility decays at rate α^i (Latency Sensitivity). TIMELYLLM generates a response in multiple segments. Hence, we represent the LLM output with a set of sequential segments

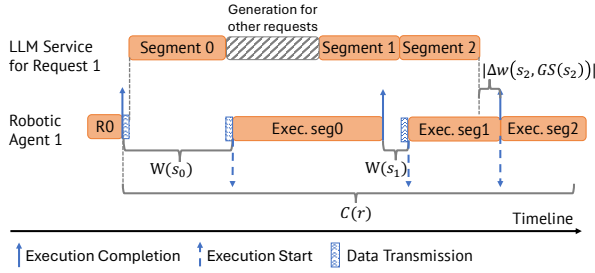


Figure 3: Three user-perceptible latencies for a robotic request: (i) Request response time $W(s_0)$: the time taken to perform the first action, which is also the waiting time of segment 0. (ii) Robot waiting time $\sum_{k=0}^K W(s_k)$, $W(s_0) + W(s_1)$ in the figure: the cumulative waiting time introduced by LLM planning, represented as the sum of waiting time for all segments. For normal LLM generation, this equals the request response time. (iii) Task completion time $C(r)$: the total duration encompassing the robot’s waiting time and execution time. We also present the execution slack time $\Delta w(s_2, GS(s_2))$ for Segment 2, as defined in Equation 3, with its generation start time denoted as $GS(s_2)$.

$[s_0, s_1, \dots, s_K]$, where K is the total number of segments. Given request r^i , the k^{th} segment of the LLM output is s_k^i . Specifically, we define the *waiting time* of a segment as the time between the completion of the preceding segment’s action, or the reception of the request in the case of the very first segment, and the start of its own action, as shown in Figure 3.

Objective function. With constrained shared resources on the edge server, TIMELYLLM aims to maximize the time utility of all received robotic requests, so that agents experience as little idle time as possible. While segmented generation improves responsiveness for the first segment, it may postpone later segments and thus increase downstream waiting time and overall task completion time. This leads to a fundamental trade-off between improving the utility of the initial response and reducing completion delay across all requests.

To capture this trade-off in a unified form, we introduce standardized notations for both initial and suspended generation. Specifically, we denote the generation time for each segment as G , the plan transmission time as C , and the execution time as $E(s_k^i)$. Table 3 defines the generation start/end time and the execution start/end time for segment s_k^i . For consistency, we define the execution end time of the initial segment as $EE(s_{-1}^i) = A(r^i) + ERT^i$, where $A(\cdot)$ represents the arrival time of each request and ERT^i denotes the expected response time (i.e., target deadline) for the initial response. Using these unified notations, the waiting time of segment s_k^i is:

$$W(s_k^i) = \max\{0, GS(s_k^i) + G + C - EE(s_{k-1}^i)\} \quad (1)$$

Crucially, any delay ($W > 0$) results in a penalty to the system’s utility. Since different agents operate with varying degrees of mission criticality (β) and sensitivity to delay (α), our objective is to maximize the total accrued time utility across all segments:

$$\max \sum_{r^i \in \mathbb{T}} \alpha^i \cdot \sum_{k=0}^{K^i} W(s_k^i) + (K^i + 1) \cdot \beta^i \quad (2)$$

Table 3: Time formulations for the generation of segment s_k^i .

Time	Formulation
Generation start time	$GS(s_k^i)$
Generation end time	$GS(s_k^i) + G$
Execution start time	$ES(s_k^i) = \max\{GS(s_k^i) + G(s_k^i) + C, EE(s_{k-1}^i)\}$
Execution end time	$EE(s_k^i) = ES(s_k^i) + E(s_k^i)$
Waiting time	$W(s_k^i) = ES(s_k^i) - EE(s_{k-1}^i) = \max\{0, GS(s_k^i) + G + C - EE(s_{k-1}^i)\}$

We prove that this segment-level formulation is equivalent to the original trade-off objective when the time utility functions (TUFs) are monotonically non-increasing (Theorem 1, proof in Appendix A). Since TUFs are monotonically non-increasing in many practical settings [55], this formulation is both theoretically justified and practically applicable.

Theorem 1 Assuming that the TUFs are monotonically non-increasing, the optimal solution to Eq. 2 is Pareto optimal with respect to both the request completion time and the first-segment time utility.

5.2 Slack-based Priority Assignment

Obtaining the optimal solution for the problem formulated in Eq. 2 is challenging, primarily due to the probabilistic nature of LLM outputs, the unpredictability of future robotic requests, and the fact that this problem is NP-hard [13]. A key challenge is that the generated robot plans are not known or predictable before generation or statically. Thus, TIMELYLLM must adjust the segment generation priority at run-time or dynamically based on the previously generated segments. Given this complexity, we employ a greedy algorithm that selects the generation with the highest priority, where the priority is continuously updated based on current slack time and task criticality. To better understand the performance gap between our algorithm and the offline optimal solution, we provide a competitive analysis in Appendix B. The analysis shows that our algorithm achieves a bounded competitive ratio (dependent on task parameters), and empirically remains close to the offline optimal, with observed ratios ranging from 1.45 to 1.90 across evaluated workloads.

Slack-based urgency modeling. Since our scheduling objective is to maximize the aggregate time utility of all segments, the priority of each segment should directly reflect how urgently it must be generated before the agent becomes idle. To capture this urgency, TIMELYLLM adopts a slack-based priority function that increases sharply as the available slack time decreases. We first define the slack time of segment s_k^i at time t as,

$$\Delta(s_k^i, t) = t + G + C - EE(s_{k-1}^i), \quad (3)$$

where $EE(s_{k-1}^i)$ is the execution (or playback) end time of the previous segment, t is the current time, G is the segment generation time, and C is the transmission time. Negative slack indicates that the generation can tolerate delay without stalling the agent, while positive slack implies that the agent is already waiting. To prioritize segments according to this urgency and task criticality, we use the following priority function:

$$\text{Pri}(s_k^i, t) = (\alpha^i \max(0, \Delta(s_k^i, t)) + \beta^i) \cdot e^{\min(0, \Delta(s_k^i, t))} + \varepsilon \cdot (-\alpha^i) \quad (4)$$

where β^i establishes the base mission criticality, and the negative coefficient α^i dictates the system's sensitivity to shrinking slack. The exponential term $e^{\min(0, \Delta)}$ driving an exponential escalation in priority as the deadline approaches. Conversely, when $\Delta > 0$, the negative coefficient α^i imposes a linear decay in priority. This mechanism effectuates a load-shedding strategy, systematically deprioritizing stalled tasks to reallocate computational resources toward viable requests.

Choice of priority parameters. We estimate the generation time G for each segment based on profiling data across a diverse set of requests. By adopting segmented generation rather than full-plan generation, our approach enables more reliable latency prediction. Empirical evaluation on a robotic application dataset (§7.3) shows that our approach reduces the Coefficient of Variation of generation time by 65.6% compared to the traditional approach (0.33 vs. 0.96).

We predict the execution time $E(\cdot)$ for each segment (i.e., robot plan) by constructing action-level time models from offline profiling. These models capture the execution time of each atomic robotic action (e.g., move forward, turn left) under varying parameters such as movement distance and turning angle. We use these estimates to compute slack for scheduling rather than to exactly predict the duration of an entire long-horizon task. Segmentation also improves stability because it shortens the prediction horizon and ties each estimate to concrete actions. The execution time for certain actions varies depending on the scenario. For instance, a search operation is faster if the target object is directly in front of the robot but slower if it is behind, requiring the robot to turn. For this situation, we estimate the minimum execution time to ensure the plan can be generated on time, even in the worst case. Since the LLM planner only invokes pre-defined atomic actions (as discussed in §2), our method readily generalizes to unseen segments and dynamic scenarios by composing the predicted execution times of individual actions. In such cases, TIMELYLLM uses conservative estimates (i.e., minimum execution time) to avoid overestimating slack during scheduling. Estimation errors may reduce the quality of priority ordering, but they do not affect the correctness of the serving framework. The scheduler continuously updates priorities after each newly generated segment reveals more concrete execution information. Because the LLM planner invokes only pre-defined atomic actions (as discussed in §2), our method generalizes to unseen segments and dynamic scenarios by composing per-action execution times.

Dynamic priority adjustment. Our scheduler updates the priority of a request in two occasions. First, whenever the serving system finishes generating a segment, the scheduler updates the priority of the corresponding request. This is because the priority function depends on segment-level parameters such as execution time (E), which are only known after generation. Second, because the priority function depends on the current time, t , the scheduler updates the priorities of all queued requests whenever it is about to select one for serving. As shown in §7.5, the overhead of updating priorities is modest. Additionally, TIMELYLLM drops

requests whose utility becomes negative and notifies the sender. This load-shedding mechanism is crucial to our theoretical analysis (Theorem 2): enforcing a lower bound on utility ($U_{\min} > 0$) ensures a bounded competitive ratio (Appendix B).

Theorem 2 (Competitive Ratio). *Assuming the system adopts an admission threshold $U_{\min} > 0$ to drop severely delayed tasks, with a batch size parameter determining the maximum block time $D_{\max}$, our algorithm is c -competitive, where its competitive ratio c is given by*

$$c = \max\left\{e^{\frac{\beta_{\max}}{|\alpha_{\max}|}}, 1 + \frac{|\alpha_{\min}| \cdot D_{\max}}{U_{\min}}\right\} \quad (5)$$

In proving this theorem, we assume that TUFs in the system belong to a finite set $\mathbb{U}$, where β^i is bounded by $\beta_{\min} \leq \beta \leq \beta_{\max}, \forall U \in \mathbb{U}$. $\alpha_{\min}$ represents the most steep negative slope in the set $\mathbb{U}$, $|\alpha_{\min}| = \max_i |\alpha^i|$. $\alpha_{\max}$ represents the least steep negative slope, in the set $\mathbb{U}$, $|\alpha_{\max}| = \min_i |\alpha^i|$. $D_{\max}$ represents the worst-case generation time for producing M segments in a single batch. We note this bound is not tight. In §7.4.6, we empirically show that the actual performance of our scheduling algorithm as compared to the offline optimum surpasses this analytically derived bound.

5.3 Latency-guided Continuous Batching

In this section, we illustrate how our system incorporates the timing requirements of robotic requests into LLM serving batching. A key challenge is managing batch size within the inference engine, as larger batches generally improve throughput but also lead to increased latency [23, 60]. Moreover, due to the variable output lengths of LLM requests [24, 63], the impact of batch size on latency is difficult to predict. To balance timing requirements with GPU utilization, our scheduler strategically controls the active batch size by deciding when to dispatch high-priority requests. While ExeGPT [36] addresses this tradeoff through simulator-scheduler interaction, it incurs high profiling overhead and runtime simulation delays. In contrast, our approach employs lightweight runtime checks to adaptively manage batching.

Our segmented scheduling approach offers an opportunity to estimate the worst-case scenario of batch influence, as the length bound for each segment is more predictable than the length of the entire LLM output, as discussed in §5.2. Leveraging this predictability, we analyze the remaining workload for the most urgent generation in the execution pool and decide whether additional requests can be safely batched without violating its timing constraint. We define the most urgent request as the one with the smallest remaining time budget, which reflects its proximity to the deadline. This budget is computed as the execution slack time minus the expected remaining generation time. The execution slack time, introduced in §5.2, is the time left until the expected response time. To conservatively bound the generation time, we use the worst-case generation time, calculated by dividing the remaining token length by the recent token generation speed. The remaining token length is the difference between the segment's estimated maximum length and the number of tokens already generated, while generation speed is derived from the recent token generation time. Beyond ensuring timely execution, we also monitor the free memory resources to ensure that the GPU memory can support the new generation. If memory availability becomes constrained, we halt the addition of

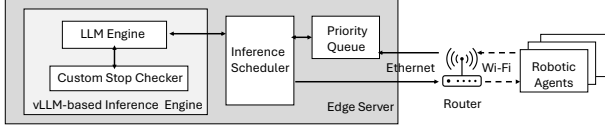


Figure 4: The prototype implementation of TIMELYLLM for robot agents. TIMELYLLM runs on the server, hosting a vLLM-based inference engine with a custom stop checker, an inference scheduler, and a priority queue to efficiently manage robot plan generation. The server communicates with agents via Ethernet and Wi-Fi through a router.

new requests to prevent memory overflows and maintain system stability.

6 Implementation

The TIMELYLLM software consists of approximately 1,500 lines of Python code, organized into three main components:

vLLM-based inference engine. We extend the LLM engine class from vLLM [3] and customize its stop checker to support segmented generation. We adapt the stopping rule in the checker to suit various robotic applications. For example, in robot control, we use regular expression matching to determine whether a robot action is present in the current LLM-generated plan. TIMELYLLM can also be implemented on programmable serving systems such as Pie [19], without modifying the underlying inference engine.

Priority queue. We adopt a priority queue to store initial and suspended generations. This queue is implemented using *PriorityQueue* and *multiprocessing*, allowing safe data sharing between processes for scheduling. We define a class to encapsulate the priority and request details for each item in the queue. When TIMELYLLM receives requests from different agents, each initial generation for the request is tagged with a request ID, agent ID, and arrival time before being added to the priority queue.

LLM inference scheduler. Our implementation employs a scheduler to queue suspended generations and retrieve them for inference. The scheduler updates the priority of requests in the queue using our priority assignment algorithm before fetching them, ensuring alignment of priorities with the current time (§5.2). As shown in §7.5, the computational overhead of these priority updates is minimal, allowing efficient request management.

7 Evaluation

We evaluate TIMELYLLM using real-world case studies for end-to-end verification, while leveraging the robotic workloads described in §7.3 for quantitative experiments. We seek to answer the following questions:

- How does TIMELYLLM perform in time sensitivity?
- How effective of each system component in enhancing TIMELYLLM’s time sensitivity?
- How effectively does TIMELYLLM generalize across diverse LLM-driven I/O limited agents and heterogeneous robotic platforms?
- How significant is the time overhead introduced by segmented generation and scheduling?

7.1 Setup

Models. Our experiments employ Llama3-8B-float16, a state-of-the-art open-source LLM series [31], which is capable of generating valid plans for robot control. In our chatbot applications, we also use Qwen2-7B-Instruct [2]. We set the inference temperature to 0 for result reproducibility.

Hardware. We deploy our prototype on an in-house edge server, equipped with an NVIDIA RTX 4090 GPU, an AMD Ryzen 9 7950X CPU, and 24 GB of host memory.

Baseline and other compared systems.

- vLLM [3] (continuous batching): This baseline employs a continuous batching strategy following an FCFS protocol to handle requests from multiple agents.
- vLLM-stream: We enhance vLLM with a streaming execution mechanism. Different from the vLLM, this approach transmits results immediately. However, it retains a run-to-completion scheduling policy. Even if the agent is busy executing the first action, the engine continues generating the subsequent tokens, locking the GPU resources.
- Other alternative variants of TIMELYLLM: We also utilize baselines derived from our system that employ different scheduling algorithms, such as First-Come, First-Served (FCFS) and Earliest Deadline First (EDF). We will provide a detailed introduction to them in §7.4.

Metrics.

- Time utility: This metric represents the time utility value of the LLM’s response. We calculate it for each task based on its actual response time. It reflects the time sensitivity of the LLM serving system.
- Response time: This metric measures the duration from receiving an agent query to the initial response (e.g., the first action in robot control), quantifying the response efficiency of the LLM serving.
- Waiting time: This metric quantifies the delay introduced by LLM planning, calculated as the difference between the task completion time and the robot’s execution time. We use this instead of task completion time to directly assess LLM system efficiency. Since robot execution time is constant for a given plan, reducing robot waiting time inherently improves task completion time.

7.2 Real-world Case Studies

7.2.1 Robot Agents. We apply TIMELYLLM to a multi-robot testbed consisting of two robot cars and a quadruped robot dog, all implementing TypeFly [11]. The robot cars are custom-built platforms powered by STM32 (for motion control), ESP32 (for video capture and wireless transmission), and equipped with front-facing cameras for object detection. The robot dog is a modified Unitree Go2 robot, using its built-in camera for visual perception. As shown in Figure 5, we construct a mock-up indoor environment of multiple search zones, each populated with various objects. We assign the robot cars lower-priority tasks: to search within their assigned zones and identify specific target objects. These tasks exhibit lower urgency and follow a TUF with gradual decay ($ERT = 1s$, $\beta = 1$,

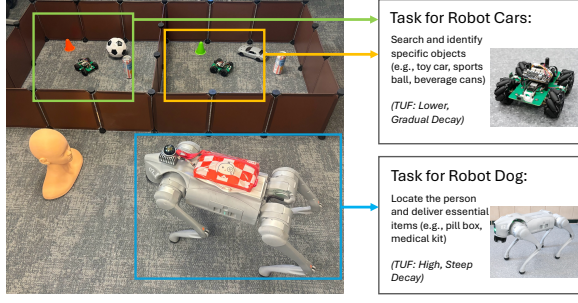


Figure 5: Real-world Testbed Setup. This setup features two robot cars conducting object search within assigned search zones, and a Unitree Go2 robot dog performing a high-priority item delivery task. All robots operate concurrently, sending requests to an edge server-hosted TIMELYLLM via Wi-Fi.

and $\alpha = -2$, following previous studies on user interaction [47] and robotics [10]). In contrast, the robot dog executes a high-priority task: locate a person (the mannequin) and deliver a medical kit, with high initial utility and steep decay ($ERT = 200ms$, $\beta = 2$, and $\alpha = -6.67$) to reflect urgency. All robots operate concurrently and send requests over Wi-Fi to TIMELYLLM hosted by an edge server (NVIDIA RTX 4090 GPU). The Wi-Fi network latency ranges from 2 to 10ms. To emulate a resource limit, we limit TIMELYLLM to processing two requests simultaneously. This setup evaluates the system’s ability to prioritize and schedule requests under real-time and resource constraints.

To evaluate performance under varying execution conditions, we design five scene configurations by altering target object placements, which affect action durations (e.g., closer objects reduce search/execution time). Each configuration is tested over three runs for both our method and the baseline, using identical placements to ensure fairness. Figure 6 presents the average results across all scene configurations. As illustrated in Figure 6a, TIMELYLLM consistently yields higher time utility than the baseline vLLM, with the most notable improvement observed for Robot Dog, where the utility increases from -2.76 to 2.00 . This gain can be attributed to TIMELYLLM’s ability to prioritize time-sensitive tasks and minimize response time. In Figure 6, we observe a substantial reduction in waiting time across all robots. For instance, the average waiting time for Robot Car 1 decreases from $697.45ms$ to $223.67ms$, and for Robot Dog from $913.88ms$ to $224.38ms$. These results underscore the effectiveness of TIMELYLLM in reducing waiting time and improving task responsiveness under varying execution dynamics.

Figure 7 illustrates the execution timeline of three concurrent robot agents, aligned to the task release time of Robot Car 2 (defined as time 0). Each robot is assigned a compound task consisting of sequential subtasks such as object search, navigation, waiting, and repositioning. Under the baseline vLLM, the FCFS policy with full-plan generation delays the robot dog. In contrast, our method uses segmented generation and utility-based scheduling, allowing the high-priority task to begin promptly while still progressing lower-priority tasks in time, yielding faster responses for all three robots.

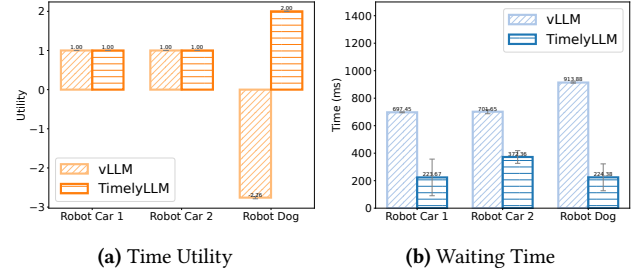


Figure 6: Performance comparison of TIMELYLLM and vLLM across varying scene configurations. The presented results average the performance across five scene configurations and three runs for each. TIMELYLLM consistently demonstrates higher time utility, particularly for Robot Dog, and significantly reduces waiting times for all robots.

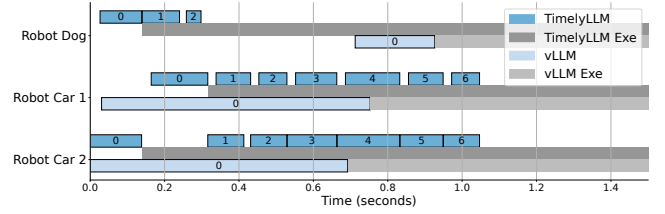


Figure 7: Time trace for scenario 1 in case study, comparing the execution timelines of three robots (Robot Dog, Robot Car 1, Robot Car 2) under our method TIMELYLLM and baseline vLLM. Blue segments represent plan generation, while gray segments represent plan execution. Our method employs utility-based scheduling, effectively prioritizing the Robot Dog request and, through segmented generation, allocates resources to Robot Car 1 for plan generation after Robot Car 2 generates its first segment, resulting in a faster response for all three robots compared to the baseline vLLM.

7.2.2 Spoken Agents. We evaluate TIMELYLLM for spoken agents by deploying an end-to-end conversational pipeline. This system architecture chains Whisper [41] for Automatic Speech Recognition (ASR), TIMELYLLM serving system for response generation, and PiperVoice [1] for Text-to-Speech (TTS) synthesis. We utilized the data trace BurstGPT [53] to model stochastic arrival times and length distributions (Figure 8). We then populated these traces with semantic content by sampling prompts from the UltraChat dataset [14] benchmark using a length mapping algorithm. We set the TUF parameters as $ERT = 1s$, $\beta = 1$, and $\alpha = -0.1$ according to prior studies on user interaction [47]. We also set the time compliance threshold to 1s for segmented generation to prevent audio fragmentation and ensure natural speech flow.

Figure 9 details the resulting performance gains. The Cumulative Distribution Function (CDF) in Figure 9a demonstrates a superior utility distribution for TIMELYLLM compared to the vLLM-stream baseline, while the baseline exhibits a long tail of negative utility values, TIMELYLLM maintains utility scores concentrated near 1 (maximum utility). Figure 9b and Figure 9c show the waveform of a representative task. As illustrated in the highlighted regions,

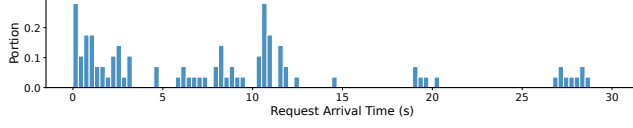


Figure 8: Data distribution for speech queries.

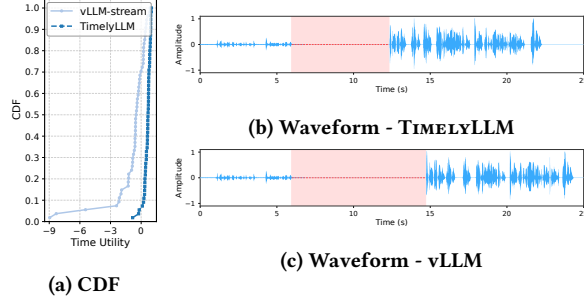


Figure 9: Performance comparison. (a) CDF of Utility. (b) and (c) show the waveforms for Task 20 under TIMELYLLM and baseline, respectively.

Table 4: Benchmark tasks used in the dataset collection.

Categories	Trace ID	Task Description
Normal request for drone	1	Look for the cat
	2	Check for keys or wallets
	3	Check the book on the cabinet
	4	Find a toy under the bed
	5	Find a tool for cutting paper
Urgent request for drone	6	Avoiding human injury
	7	Preventing potential collision
	8	Evading animal pursuit
Complex request for robot arm	9	Object stacking
	10	Desk cleaning
	11	Fruit and vegetable classification
Request for quadruped robot	12	Find the household item
	13	Guide to the door
	14	Bark to deter intruders

TIMELYLLM (Figure 9b) initiates the audio stream significantly earlier (2.47s). In contrast, the vLLM baseline (Figure 9c) exhibits a prolonged silence period. This reduction confirms that TIMELYLLM effectively resolves the resource contention to mitigate latency. Besides, the waveform structure of TIMELYLLM remains consistent with that of the baseline, showing no signs of fragmentation.

7.3 Benchmark Construction

To evaluate multi-agent serving performance, we developed LR-Trace, a dataset collection system grounded in real-world robotic profiles. We profiled execution latencies on three physical platforms: a Ryze Tello drone, a Neuromeka Indy7 arm, and a Unitree Go2 quadruped (Figure 10). Based on these profiles, we constructed 14 diverse benchmark tasks (Table 4) ranging from simple navigation to complex reasoning, using Llama3-8B to generate valid plans. LRTrace composes these tasks into programmable workloads using a Poisson arrival process to mimic real-world traffic. We define five

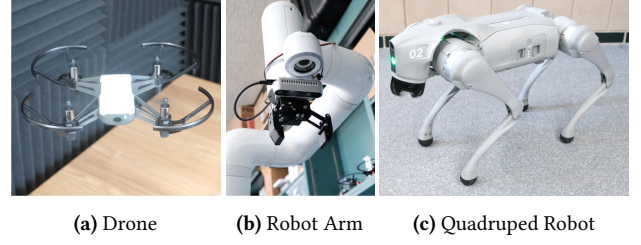


Figure 10: Robots used in data collection: We utilize a Ryze Tech Tello drone [50], a Neuromeka Indy7 Pro robotic arm [34], and a Unitree Go2 Pro quadruped robot dog [51]. These platforms are used to profile the actual execution time of various robotic skills.

Table 5: Different Workloads for Evaluation. (WID: Workload ID, EPS: Events Per Second, Max TPE: Maximum Task number Per Event, AN: Agent Number, RS: Robotic System). WID 1 incorporates tasks with varying TUF characteristics. WID 2 differs from the first in that it offers a higher degree of task parallelism. WID 3 is designed for more complex tasks involving the robot arm. WID 4 differs from WID 3 in robotic systems. WID 5 focuses on mixed robot tasks.

WID	Categories	EPS	Max TPE	Trace ID	AN	Time (s)	RS	Robots
1	Mixed TUF	0.25	8	1-8	25	260	[11]	Fig. 10a
2	High-Parallel	0.25	16	1-8	42	300	[11]	Fig. 10a
3	Arm - TypeFly	0.1	8	9-11	40	900	[11]	Fig. 10b
4	Arm - FLTRNN	0.5	8	9-11	81	100	[61]	Fig. 10b
5	Mixed Robots	0.25	12	3-8,12+	47	500	[11]	Fig. 10c, 10a

specific workloads (Table 5) with varying degrees of parallelism and urgency for our evaluation.

For tasks identified with trace IDs [1-5] and [9-14], we adopt the TUF configuration of normal robot tasks, where the ERT is 1s, $\beta = 1$, and $\alpha = -2$. This α value is derived assuming a cut-off time of 1.5 seconds, beyond which the utility becomes negative. Conversely, for tasks with trace IDs [6-8], which are designated as urgent, the TUF configuration is adjusted to an ERT of 200ms, β at 2, and α at -6.67 , with the α value computed using a cut-off time of 0.5s.

7.4 Overall Performance of TIMELYLLM

In this section, we evaluate the overall performance of TIMELYLLM on the dataset introduced in §7.3.

7.4.1 Performance with different workloads. Figure 11 presents various performance metrics under different workloads. For the evaluated workloads, we use WID1 as low workloads and WID2 as high workloads, as described in §7.3. The high-workload setting includes workloads with high request concurrency, with up to 16 parallel tasks per event. We simulate an 8ms network latency by measuring the time each response remains in the execution queue and adding necessary delays. This latency is based on data transmission tests conducted between two devices on a local network.

As shown in Figure 11, we measure and record the time utility, response time, and the waiting time achieved by both TIMELYLLM and the baseline. The recorded results are averaged based on the task

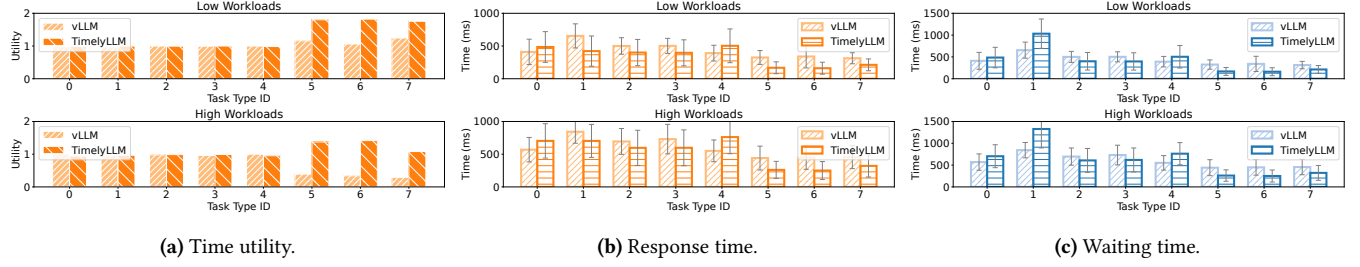


Figure 11: End-to-end performance under different levels of resource contention. Based on (a), *TIMELYLLM* significantly improves utility for urgent requests compared to the *vLLM* baseline. Furthermore, the remaining figures show that *TIMELYLLM* enhances time utility primarily by reducing response time, while minimizing idle gaps between consecutive planning segments.

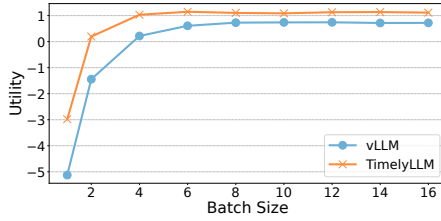


Figure 12: Performance of *TIMELYLLM* with different levels of throughput (batch size).

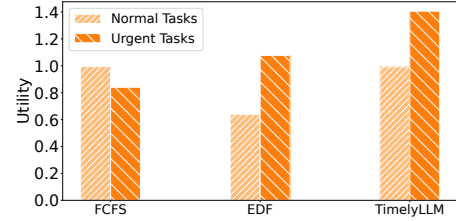


Figure 13: Performance of *TIMELYLLM* with different scheduling policies, i.e., FCFS, EDF.

type of each request. Task types 0-4 are normal requests, and Task types 5-8 are urgent requests. Our method achieves the maximum utility for normal tasks, and 89.50% of the maximum time utility for urgent tasks. In contrast, the baseline *vLLM* achieves only 57.67% of the maximum time utility for urgent tasks. This improvement is attributed to our system’s ability to respond to urgent tasks within a shorter duration (Figure 11). The lower part of Figure 11a depicts the time utility under conditions of heightened resource contention. Although utility declines due to significant resource contention, our method still improves the total time utility by 1.47 $\times$ compared with the baseline. From the upper part of Figure 11b and Figure 11c, we observe that *TIMELYLLM* maintains a nearly consistent waiting time with the response time. This indicates that for most tasks, our method successfully generates the next segment before the execution of the previous segment, thereby minimizing the number of waiting periods between consecutive segments.

7.4.2 Performance with different throughput. We now evaluate the performance of *TIMELYLLM* across various throughput levels by fixing the maximum batch size rather than employing our adaptive batching method, while the other components of *TIMELYLLM* remain unchanged. We similarly adjust the baseline batch size, capping the number of added requests below a fixed maximum. In this experiment, we didn’t filter out negative utility requests to fully illustrate the impact of multi-task interference on utility. To assess a broad spectrum of batch sizes, we select WID2 for our evaluation, which has up to 16 tasks per event. As shown in Figure 12, we demonstrate the time utility achieved by both our method and the baseline across these varying batch sizes. It shows that the utility values for both models initially increase sharply with the batch size.

Upon reaching a batch size of 6, *TIMELYLLM* achieves its highest utility at 1.15, whereas *vLLM* reaches its peak utility of 0.74 at a batch size of 12. Beyond these points, there is a slight decline in utility for both models. This is because interference between requests grows with batch size. When the batch size exceeds GPU memory limitations, further increases may not increase the actual number of running generations. Overall, *TIMELYLLM* consistently outperforms *vLLM* across all tested batch sizes.

7.4.3 Performance with different scheduling algorithms. We evaluate the performance of our scheduling algorithm by comparing it to segmented generation with FCFS and EDF algorithms. Each baseline employs our segmented generation approach, maintaining the initial generation priority for suspended generations. From Figure 13, we observe that EDF prioritizes urgent requests but neglects normal tasks due to its preference for shorter deadlines. Conversely, FCFS maximizes the utility of normal tasks but performs poorly with urgent tasks. In contrast, *TIMELYLLM* achieves better time utility for urgent tasks (167.36% relative to FCFS and 130.49% relative to EDF) without compromising the utility of normal tasks, thanks to its efficient prioritization approach based on the scaled utility density.

7.4.4 Performance with different robotic systems. We evaluate generalizability using robot arm tasks across two open-source robotic systems: TypeFly [11], characterized by its concise syntax, and FLTRNN [61], designed for long-horizon task planning. This experiment uses the WID3 and WID4 workloads. As shown in Figure 14a and Figure 14b, *TIMELYLLM* substantially enhances the time utility of robot arm tasks. On average, *TIMELYLLM* improves utility by 1.52 and reduces waiting time by 84.53% compared to *vLLM*, and

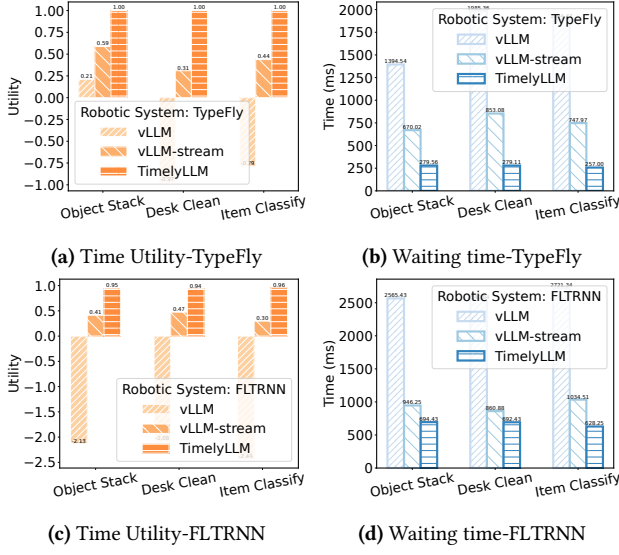


Figure 14: Performance on robot arm with two robotic systems (i.e., TypeFly [11] and FLTRNN [61]).

Table 6: The segmented overhead across eight traces, presented in milliseconds and as a percentage.

Trace ID	1	2	3	4	5	6	7	8
Overhead (ms)	13.7	24.1	19.9	19.9	16.0	13.1	14.4	14.8
Overhead (%)	3.0	2.6	5.6	2.8	3.2	3.7	4.1	4.2

increases utility by 0.55 while reducing waiting time by 64.08% relative to vLLM-stream. This is because the longer response/plan provides more segmentation opportunities, thereby gaining more benefits. We also observe that vLLM-stream performs better than vLLM but results in inferior performance compared to TIMELYLLM. This is because vLLM-stream generates the whole plan at once and does not free up resources for other urgent requests. In contrast, TIMELYLLM effectively halts requests that already have partial plans for execution, reallocating resources to urgent requests.

In the FLTRNN system (Figure 14c and Figure 14d), we observe a similar trend: TIMELYLLM maintains superior time utility over vLLM-stream and vLLM. However, we also note that waiting times in FLTRNN are generally higher than in TypeFly. This is because TypeFly employs a more concise language for robot plan generation.

7.4.5 Performance with mixed robot types. We evaluate the capability of our system to concurrently support requests from multiple robot types using WID 5 as the test workload. This workload comprises three normal tasks and three urgent tasks from drones, along with three tasks from quadruped robots. As shown in the Figure 15, all three methods achieve the maximum utility for quadruped robots. However, our approach attains the highest overall utility, encompassing both drone and quadruped robot tasks. These results underscore the effectiveness of our system in accommodating heterogeneous robot requests.

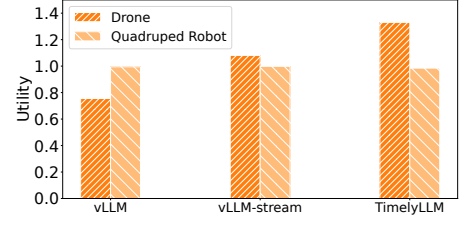


Figure 15: Performance in robotic applications involving mixed robot types (i.e., drone and quadruped robots).

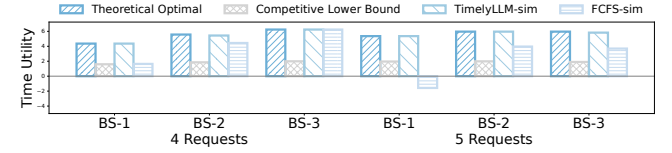


Figure 16: Performance gap between TIMELYLLM and the theoretical optimum under two workloads (i.e., 4 requests and 5 requests) with varying batch sizes (e.g., BS-2: Batch Size = 2).

Table 7: The scheduling overhead for various sizes of task queues.

Task Number	1	2	4	6	8
Scheduling Overhead (ms)	0.58	0.83	1.25	1.58	2.03

7.4.6 Optimality gap analysis. To better understand the performance gap between our algorithm and the offline optimal, we conduct both competitive analysis (as mentioned in §5.2) and empirical evaluations. We obtain the theoretically optimal solution through a brute-force search that explores all possible scheduling decisions. We compute the competitive lower bound by dividing the optimal utility by the theoretical competitive ratio derived in Theorem 2 with $U_{min} = 0.5$. For comparison, we evaluate our scheduling algorithm against this bound as well as the FCFS baseline. Our evaluation is based on tasks from WID 2. As shown in Figure 16, we vary the batch size limits (1, 2, and 3), yielding competitive ratios of 2.73, 3.00, and 3.13, respectively (computed via Equation 8). Across all configurations, our algorithm empirically outperforms both the FCFS baseline and the computed competitive lower bounds.

7.5 System Overhead

7.5.1 Segmented generation overhead. We assess the overhead of segmented generation by conducting separate tests for normal and segmented generation. The overhead specific to the segmented generation is determined by subtracting the time taken for the normal generation from that of the segmented generation. This overhead includes stop-checking and caching/retrieval for generation contexts. As shown in Table 6, the segmented overhead differs among tasks due to the different response content, but all of them remain below 6% of the total segmented generation time.

7.5.2 Scheduling overhead. With our scheduling algorithm, we need to calculate the TUF for each task when updating the priority. This occurs every time we retrieve a request from the queue.

We quantify the scheduling overhead by measuring the duration required to update the priority and fetch the request. From Table 7, we observe an increase in scheduling overhead with the number of tasks. However, it remains minimal, with only about 2 ms cost for eight tasks.

8 Related work

LLM request scheduling. State-of-the-art LLM serving systems [3, 6, 35, 59] typically implement FCFS continuous batching and do not consider the timing semantics of downstream agents. Recent advancements [24, 36, 37, 40, 45, 56] refine LLM scheduling to improve queueing efficiency, output-length awareness, or SLO compliance. For instance, FastServe [56] employs a multi-level feedback queue scheduler that prioritizes jobs based on their cumulative generated length. S3 [24] optimizes scheduling by predicting the output sequence length. Another study [16] schedules requests by estimating their predicted relative ranks of output lengths. Andes [28] exploits the gap between token generation and user reading/listening speed in text-streaming services, and dynamically preempts requests at token granularity to improve QoE. Overall, these works optimize request- or token-level scheduling for text streaming or general LLM serving. In contrast, TIMELYLLM supports time-sensitive agent planning and execution for Physical-I/O limited agents by scheduling semantically executable segments, and by dynamically pausing and resuming generation based on generated content and agent-side execution.

Most related to TimelyLLM, Semantic Scheduling [20] uses input semantics to predict request urgency and prioritize requests accordingly. Although it supports request-level preemption, its scheduling decisions are not driven by semantics revealed during generation. In contrast, TimelyLLM supports generation-level pause and resume, and updates urgency online based on revealed segment content and agent-side execution slack. For example, after a partial plan is generated and execution begins, continuing that generation may become temporarily less urgent, allowing compute to be shifted to other tasks with tighter slack.

LLM with external executors. LLMs are increasingly integrated with external executors, encompassing both software (e.g., auxiliary AI models [46]) and hardware (e.g., robotic systems [11]), to tackle complex tasks. LLMs with function call capabilities are widely employed to enable mobile task automation [27, 54] and solve complicated AI tasks [46]. Our contribution to scheduling supports both non-blocking and blocking function calls, where each function call represents an executable skill in our system. To handle blocking function calls, which typically depend on return values to guide subsequent generation [5, 18], our system can support it by utilizing a waiting queue. Our scheduler could temporarily place the blocked requests in the waiting queue and reinsert them into the priority queue with updated priority when the function is completed. Another line of research enables the asynchronous execution of LLM-generated plans [11, 25], streaming outputs directly to executors. While this method is effective in environments with unlimited computational resources, it struggles in scenarios where

multiple requests contend for resources. Our evaluation (vLLM-stream) compares this approach, where our method demonstrates superior performance.

9 Concluding Remarks

In this paper, we present TIMELYLLM, a time-sensitive LLM serving system designed for multiple physical I/O limited agents. TIMELYLLM enables the segmented generation of responses and dynamically schedules each request in accordance with time-sensitive requirements. We implement TIMELYLLM on an open-source LLM serving framework and conduct evaluations across various applications. Our findings demonstrate that TIMELYLLM outperforms existing state-of-the-art solutions for LLM serving, delivering superior time utility and reduced waiting time when handling multiple real-world queries. We next discuss the broader applicability of TIMELYLLM and its deployment implications.

Multi-GPU extension. The computation of LLMs necessitates distributed processing on multiple GPUs when the model parameters exceed the memory capacity of a single GPU. TIMELYLLM is compatible with two common multi-GPU deployment modes. In data parallelism, each GPU replica maintains its own ready queue and suspended states, allowing TIMELYLLM to operate independently on each replica with only standard load balancing. In tensor or pipeline parallelism, TIMELYLLM places the scheduler at the head node and reuses the existing model-parallel runtime for execution and resumption. Because it introduces no inter-GPU communication beyond that already required by model parallelism, TIMELYLLM adds little coordination overhead in multi-GPU settings.

Scope and Applicability. The effectiveness of TIMELYLLM depends on exploitable slack and system contention. Under high concurrency, GPU contention becomes a primary bottleneck, and reclaiming compute from slack-rich requests can yield substantial gains. Although dynamic reprioritization introduces overhead from segmentation, suspension, and resumption, §7.5 shows that this cost is negligible relative to the hundreds of milliseconds typically spent on decoding, supporting scalability under online concurrency. In §7.4, TIMELYLLM consistently outperforms the baseline with up to 16 concurrent tasks per event.

For workloads with tight feedback loops, such as ReAct-style planning [58], exploitable slack may be limited because later generation depends on execution feedback. As a result, TIMELYLLM may provide smaller gains than in slack-rich workloads. Therefore, TIMELYLLM is most effective for slack-rich, Physical-I/O limited scenarios.

Acknowledgments

This work is supported in part by National Science Foundation (NSF) Athena AI Institute (Award #2112562) and Yale University. The authors thank the reviewers and the paper shepherd for their constructive feedback.

References

- [1] 2023. Piper: a fast, local neural text-to-speech system. <https://github.com/rhasspy/piper>.
- [2] 2024. Qwen2 technical report. (2024).
- [3] 2024. vLLM: easy, fast, and cheap LLM serving for everyone.
- [4] Marah Abidin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadallah, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, et al. 2024. Phi-3 technical report: a highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219* (2024).
- [5] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. 2025. InferCept: efficient intercept support for augmented large language model inference. In *Proc. ICML*.
- [6] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *Proc. USENIX OSDI*.
- [7] Siddhant Arora, Kai-Wei Chang, Chung-Ming Chien, Yifan Peng, Haibin Wu, Yossi Adi, Emmanuel Dupoux, Hung-Yi Lee, Karen Livescu, and Shinji Watanabe. 2025. On the landscape of spoken language models: a comprehensive survey. *arXiv preprint arXiv:2504.08528* (2025).
- [8] Michal Barčíš and Hermann Hellwagner. 2019. An evaluation model for information distribution in multi-robot systems. In *Proc. IEEE Conf. Computer Communications Workshops (INFOCOM WKSHPS)*.
- [9] Tom B Brown. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).
- [10] Adrian Carrio, Sai Vemprala, Andres Ripoll, Srikanth Saripalli, and Pascual Cam-poy. 2018. Drone detection using depth maps. In *Proc. IEEE/RSJ IROS*.
- [11] Guojun Chen, Xiaojing Yu, Neiven Ling, and Lin Zhong. 2024. TypeFly: flying drones with large language model. *arXiv preprint arXiv:2312.14950v2* (2024).
- [12] Guojun Chen, Xiaojing Yu, Neiven Ling, and Lin Zhong. 2025. Chatfly: low-latency drone planning with large language models. *IEEE Transactions on Mobile Computing* (2025).
- [13] Ken Chen and Paul Muhlethaler. 1996. A scheduling algorithm for tasks described by time value function. *Real-Time Systems* 10 (1996), 293–312.
- [14] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2023. Enhancing chat language models by scaling high-quality instructional conversations. *arXiv:2305.14233 [cs.CL]*
- [15] Hugging Face. 2024. *Transformers: state-of-the-art machine learning for PyTorch, TensorFlow, and JAX*.
- [16] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM scheduling by learning to rank. *arXiv preprint arXiv:2408.15792* (2024).
- [17] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt cache: modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems* 6 (2024), 325–338.
- [18] In Gim, Seung-seob Lee, and Lin Zhong. 2024. Asynchronous LLM function calling. *arXiv preprint arXiv:2412.07017* (2024).
- [19] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. 2025. Pie: a programmable serving system for emerging LLM applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 415–430.
- [20] Rongjie Huang, Mingze Li, Dongchao Yang, Jiatong Shi, Xuankai Chang, Zhenhui Ye, Yuning Wu, Zhiqing Hong, Jiawei Huang, Jinglin Liu, et al. 2024. AudioGPT: understanding and generating speech, music, sound, and talking head. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 23802–23804.
- [21] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. GPT-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [22] Raisa Islam and Owana Marzia Moushi. 2025. GPT-4o: the cutting-edge advancement in multimodal LLM. In *Intelligent Computing-Proceedings of the Computing Conference*. Springer, 47–60.
- [23] Zhehao Jiang, Neiven Ling, Xuan Huang, Shuyao Shi, Chenhao Wu, Xiaoguang Zhao, Zhenyu Yan, and Guoliang Xing. 2023. CoEdge: a cooperative edge system for distributed real-time deep learning tasks. In *Proc. IEEE/ACM IPSN*.
- [24] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S³: increasing GPU utilization during generative inference for higher throughput. In *Proc. NeurIPS*.
- [25] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. 2023. An LLM compiler for parallel function calling. *arXiv preprint arXiv:2312.04511* (2023).
- [26] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [27] Sunjae Lee, Junyoung Choi, Jungjae Lee, Munim Hasan Wasi, Hojun Choi, Steven Y Ko, Sangeun Oh, and Insik Shin. 2024. MobileGPT: augmenting LLM with human-like app memory for mobile task automation. (2024).
- [28] Jiachen Liu, Jae-Won Chung, Zhiyu Wu, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. 2024. Andes: defining and enhancing quality-of-experience in LLM-based text streaming services. *arXiv preprint arXiv:2404.16283* (2024).
- [29] Zeyi Liu, Arpit Bahety, and Shuran Song. 2023. Reflect: summarizing robot experiences for failure explanation and correction. *arXiv preprint arXiv:2306.15724* (2023).
- [30] David P Maynard, Samuel E Shipman, Raymond K Clark, J Duane Northcutt, Russell B Kegley, Betsy A Zimmerman, and Peter J Keleher. 1988. An example real-time command, control, and battle management application for alpha. *Archons Project TR-88121, CMU* (1988).
- [31] Meta. 2024. The Llama 3 herd of models. <https://ai.meta.com/research/publications/the-llama-3-herd-of-models/>.
- [32] Meta. 2024. Meta AI assistant built with LLaMA 3. <https://about.fb.com/news/2024/04/meta-ai-assistant-built-with-llama-3/>.
- [33] Neawhen. 2026. TimelyLLM: time-sensitive LLM serving system for physical-I/O limited agents. <https://github.com/Neawhen/TimelyLLM>. Accessed: 2026-04-11.
- [34] Neuromeka. 2024. *Neuromeka Indy*.
- [35] NVIDIA. 2024. TensorRT-LLM. <https://nvidia.github.io/TensorRT-LLM/overview.html>.
- [36] Hyungjun Oh, Kihong Kim, Jaemin Kim, Sungkyun Kim, Junyeol Lee, Du-seong Chang, and Jiwon Seo. 2024. ExeGPT: constraint-aware resource scheduling for LLM inference. In *Proc. ACM ASPLOS*.
- [37] Archit Patke, Dharmath Reddy, Saurabh Jha, Haoran Qiu, Christian Pinto, Shengkun Cui, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2024. One queue is all you need: resolving head-of-line blocking in large language model serving. *arXiv preprint arXiv:2407.00047* (2024).
- [38] Hannah Pelikan and Emily Hofstetter. 2023. Managing delays in human-robot interaction. *ACM Transactions on Computer-Human Interaction* (2023).
- [39] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivan Agha, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems* 5 (2023), 606–624.
- [40] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew T Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. 2024. Efficient interactive LLM serving with proxy model-based sequence length prediction. *arXiv preprint arXiv:2404.08509* (2024).
- [41] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust speech recognition via large-scale weak supervision. In *International conference on machine learning*. PMLR, 28492–28518.
- [42] Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Suenderhauf. 2023. Sayplan: grounding large language models using 3D scene graphs for scalable task planning. *arXiv preprint arXiv:2307.06135* (2023).
- [43] Allen Z Ren, Anushri Dixit, Alexandra Bodrova, Sumeet Singh, Stephen Tu, Noah Brown, Peng Xu, Leila Takayama, Fei Xia, Jake Varley, et al. 2023. Robots that ask for help: uncertainty alignment for large language model planners. *arXiv preprint arXiv:2307.01928* (2023).
- [44] Shireen Seakhoa-King, Paul Balaji, Nicolas Trama Alvarez, and William J Knotenbelt. 2019. Revenue-driven scheduling in drone delivery networks with time-sensitive service level agreements. In *Proc. EAI Int. Conf. Performance Evaluation Methodologies and Tools*.
- [45] Rana Shahout, Eran Malach, Chunwei Liu, Weifan Jiang, Minlan Yu, and Michael Mitzenmacher. 2024. Don't Stop Me Now: embedding based scheduling for LLMs. *arXiv preprint arXiv:2410.01035* (2024).
- [46] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2024. HuggingGPT: solving AI tasks with ChatGPT and its friends in Hugging Face. In *Proc. NeurIPS*.
- [47] Ben Shneiderman. 1984. Response time and display rate in human performance with computers. *ACM Computing Surveys (CSUR)* (1984).
- [48] Rohan Sinha, Amine Elhafi, Christopher Agia, Matthew Foutter, Edward Schmerling, and Marco Pavone. 2024. Real-time anomaly detection and reactive planning with large language models. *arXiv preprint arXiv:2407.08735* (2024).
- [49] Marta Skreta, Naruki Yoshikawa, Sebastian Arellano-Rubach, Zhi Ji, Lasse Bjørn Kristensen, Kourosh Darvish, Alan Aspuru-Guzik, Florian Shkurti, and Animesh Garg. 2023. Errors are useful prompts: instruction guided task programming with verifier-assisted iterative prompting. *arXiv preprint arXiv:2303.14100* (2023).
- [50] Tello. 2023. *Tello SDK user guide*. <https://dlcdn.rzerobotics.com/downloads/Tello/Tello%20SDK%202.0%20User%20Guide.pdf>
- [51] Uniree. 2024. *Uniree Go2*.
- [52] Shu Wang, Muzhi Han, Ziyuan Jiao, Zeyu Zhang, Ying Nian Wu, Song-Chun Zhu, and Hangxin Liu. 2024. LLM³: large language model-based task and motion planning with motion failure reasoning. *arXiv preprint arXiv:2403.11552* (2024).
- [53] Yuxin Wang, Yuhang Chen, Zeyu Li, Xueze Kang, Yuchuan Fang, Yeyu Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, et al. 2025. BurstGPT: a real-world workload dataset to optimize LLM serving systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 5831–5841.

- [54] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. AutoDroid: LLM-powered task automation in Android. In *Proc. ACM MobiCom*.
- [55] Wikipedia contributors. 2024. Time-utility function — Wikipedia. https://en.wikipedia.org/wiki/Time-utility_function
- [56] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuazhe Liu, and Xin Jin. 2023. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920* (2023).
- [57] Haibin Wu, Yuxuan Hu, Ruchao Fan, Xiaofei Wang, Kenichi Kumatani, Bo Ren, Jianwei Yu, Heng Lu, Lijuan Wang, Yao Qian, et al. 2025. Towards efficient speech-text jointly decoding within one speech language model. *arXiv preprint arXiv:2506.04518* (2025).
- [58] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. ReAct: synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.
- [59] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: a distributed serving system for transformer-based generative models. In *Proc. USENIX OSDI*.
- [60] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. 2023. SHEPHERD: serving DNNs in the wild. In *Proc. USENIX NSDI*.
- [61] Jiatao Zhang, Lanling Tang, Yufan Song, Qiwei Meng, Haofu Qian, Jun Shao, Wei Song, Shiqiang Zhu, and Jason Gu. 2024. FLTRNN: faithful long-horizon task planning for robotics with large language models. In *Proc. IEEE ICRA*.
- [62] Jesse Zhang, Jiahui Zhang, Karl Pertsch, Ziyi Liu, Xiang Ren, Minsuk Chang, Shao-Hua Sun, and Joseph J Lim. 2023. Bootstrap your own skills: learning to solve new tasks with large language model guidance. *arXiv preprint arXiv:2310.10021* (2023).
- [63] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. 2024. Response length perception and sequence scheduling: an LLM-empowered LLM inference pipeline. In *Proc. NeurIPS*.

Appendix

A Proof of problem equivalence

As formulated in Equation 6, our objective is maximizing the time utility of response time while minimizing the task completion time across all requests. Specifically, the response time of a request $W(r^i)$ is represented by the waiting time of its first segment, $W(s_0^i)$. Now, we proceed with the proof of Theorem 1.

$$\begin{aligned} & \max \sum_{r^i \in \mathbb{T}} U^i(W(s_0^i)) \\ & \min \sum_{r^i \in \mathbb{T}} C(r^i) \\ & C(r^i) = \sum_{k=0}^{K^i} (W(s_k^i) + E(s_k^i)) \end{aligned} \quad (6)$$

Theorem 1 Assuming that the TUFs are monotonically non-increasing, the optimal solution to Equation 2 is Pareto optimal with respect to both the request completion time and the first-segment time utility.

Lemma 1. For any request r^i and segment s_k^i , $U^i(W(s_k^i))$ is monotonically non-increasing with respect to $W(s_k^i)$. Formally, if $W(s_k^i) \leq W'(s_k^i)$, then:

$$U^i(W(s_k^i)) \geq U^i(W'(s_k^i))$$

Lemma 2. Under the optimal scheduling solution to Eq. 2, no request completion time can be further reduced.

Proof. Let x^* be the optimal scheduling solution to Eq. 2.

$$x^* = \arg \max_x \sum_{r^i \in \mathbb{T}} (U_0^i(W(s_0^i)) + \sum_{k=1}^{K^i} U_1^i(W(s_k^i)))$$

We denote the objective value achieved by the scheduling solution x as $v(x)$,

$$v(x) = \sum_{r^i \in \mathbb{T}} (U_0^i(W(s_0^i; x)) + \sum_{k=1}^{K^i} U_1^i(W(s_k^i; x)))$$

Suppose there exists another solution x' that improves the completion time of at least one request r^j without impacting the total time utility of the first segments:

$$C(r^j; x') < C(r^j; x^*),$$

Since $C(r^j) = \sum_{k=0}^{K^j} (W(s_k^j) + E(s_k^j))$ and execution time $\sum_{k=0}^{K^j} E(s_k^j)$ is a constant, any reduction in $C(r^j)$ implies a reduction in waiting time for at least one segment $s_k^j(k>0)$:

$$W(s_k^j; x') < W(s_k^j; x^*)$$

Since no other requests need to be negatively impacted by this change, we have:

$$U_1^j(W(s_k^j; x')) \geq U_1^j(W(s_k^j; x^*))$$

This inequality should be strict if there is a strict waiting time reduction:

$$U_1^j(W(s_k^j; x')) > U_1^j(W(s_k^j; x^*))$$

Thus, we have:

$$v(x') > v(x^*)$$

If x^* is truly optimal for Eq. 2, there could be no feasible solution x' that yields a strictly higher objective value. But we have just shown that by reducing one request's completion time, we can increase the overall U of all segments' waiting time. This contradicts the assumption that x^* is optimal. Hence, we prove that under the optimal solution x^* , no request completion time can be further reduced.

Lemma 3. Under the optimal scheduling solution to Eq. 2, no time utility of the first segment can be increased.

Proof. Let x^* be the optimal scheduling solution to Eq. 2. Suppose there exists another solution x' that increases the time utility of the first segment for at least one request r^j :

$$U_0^j(W(s_0^j; x')) > U_0^j(W(s_0^j; x^*))$$

We denote the objective value achieved by the scheduling solution x as $v(x)$,

$$v(x) = \sum_{r^i \in \mathbb{T}} (U_0^i(W(s_0^i; x)) + \sum_{k=1}^{K^i} U_1^i(W(s_k^i; x)))$$

Since the waiting time of other segments is not impacted by this change (no negative impact on task completion time) and there is no negative impact on other requests, we have:

$$v(x') > v(x^*)$$

We have achieved a higher total TUF value with solution x' , contradicting optimality. Hence, our initial assumption is false, and it must be that under the optimal solution for Eq. 2, no time utility of the first segment can be further increased without violating optimality.

Conclusion: We have shown that under the optimal solution of Eq. 2, no request completion time can be further reduced and no time utility of the first segment can be increased. These are exactly

the two conditions imposed by Eq. 6. Hence, the optimal scheduling solution to Eq. 2 is Pareto optimal with respect to Eq. 6.

B Competitive Ratio Analysis

Definition of Competitive Ratio. We define a scheduling algorithm x to be a sequence of segment generation time $[S(s)]$. We define the optimal offline scheduling algorithm as x^* . x^* is capable of maximizing time utility under the assumption that the arrival time of all requests, the generation and execution time of all segments, and the characteristics of all time utility functions are known a priori. We denote the value of a scheduling algorithm as $v(x)$, which is the overall time utility achieved by this algorithm, denoted as

$$v(x) = \sum_{r^i \in \mathbb{T}} \alpha^i \cdot \sum_{k=0}^{K^i} W(s_k^i) + (K^i + 1) \cdot \beta^i$$

We define that scheduling algorithm x is strictly c -competitive if for any request stream we have:

$$c \cdot v(x) \geq v(x^*) \quad (7)$$

Theorem 2 (Competitive Ratio). Assuming the system adopts an admission threshold $U_{min} > 0$ to drop severely delayed tasks, with a batch size parameter determining the maximum block time D_{max} , our algorithm is c -competitive, where its competitive ratio c is given by

$$c = \max\{e^{\frac{\beta_{max}}{|\alpha_{max}|}}, 1 + \frac{|\alpha_{min}| \cdot D_{max}}{U_{min}}\} \quad (8)$$

We suppose that each TUF in the system belongs to a finite set $\mathbb{U}$, where β^i is bounded by $\beta_{min} \leq \beta \leq \beta_{max}$, $\forall U \in \mathbb{U}$. α_{min} represents the most steep negative slope in the set $\mathbb{U}$, $|\alpha_{min}| = \max_i |\alpha^i|$. α_{max} represents the least steep negative slope, in the set $\mathbb{U}$, $|\alpha_{max}| = \min_i |\alpha^i|$.

Proof. We divide the entire timeline into non-overlapping intervals, each interval is a fixed duration D_{max} . D_{max} represents the worst-case generation time for producing M segments in a single batch. So that we can ensure at least one segment begins generating during each time interval whenever there are unfinished tasks.

$$D_{max} = \max_{1 \leq m \leq M} D(m) = D(M)$$

Then each interval can be represented as

$$I_j = [j \cdot D_{max}, (j+1) \cdot D_{max}), \quad j = 0, 1, 2, \dots, J$$

- Let O_j : be the set of all segments completed by the offline optimal algorithm x^* within the interval I_j .
- Let A_j : be the set of all segments completed by our online algorithm x within the interval I_j .

For each interval, the overall time utility achieved by offline optimal algorithm and our online algorithm can be represented as,

$$v(x^*) = \sum_{s^* \in O_j} U(W(s^*); x^*), \quad v(x) = \sum_{s \in A_j} U(W(s); x)$$

We assume that a segment generated at any time within the same interval has the same utility. We then try to charge the offline algorithm's utility to the online algorithm by a bounded factor. For each j -th interval, there are two main cases:

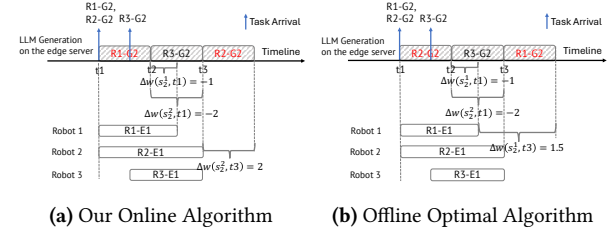


Figure 17: An example for C2: At time t_1 , both $R1-G2$ and $R2-G2$ arrive, our algorithm selects $R1-G2$ for generation since it has less slack time $|\Delta w|$. However, the optimal schedule is in the opposite order. This is because $R3-G2$ arrives later, which blocks the generation of $R2-G2$. While $R1-G2$ and $R2-G2$ possess the same initial utility, $R2-G2$ exhibits a higher degradation rate $|\alpha|$. Consequently, prioritizing the execution of $R2-G2$ remains the optimal strategy to maximize the time utility. ($R1: \alpha = -1, \beta = 1$, $R2: \alpha = -2, \beta = 1$, $R3: \alpha = -1, \beta = 2$, $G=2$, batch size=1, R: request, G: generation, E: execution, Δw : slack time.). A key takeaway is that myopically selecting tasks based on slack time can lead to suboptimal scheduling, as our online algorithm cannot anticipate future tasks, potentially causing blockages; when blockages occur and task utility declines, the degradation rates become critical, with the extent of utility loss depending on the blocking time (batch size) and the maximum degradation rate (α_{min}) within the task set.

Case 1: offline optimal algorithm x^* and our online algorithm x choose the same segment. In this case, $s \in O_j \cap A_j$,

$$U(W(s^*); x^*) = U(W(s); x)$$

Case 2: offline optimal algorithm x^* and our online algorithm x choose different segments. Let x^* choose segment(s) s^* , while x choose segment(s) s . In this case, the reason our online scheduling algorithm does not select the segment s^* can be attributed to one of the following two conditions:

- C1. Segment s^* chosen by the offline optimal algorithm has already been generated by the online algorithm in previous intervals.
- C2. Segment s^* arrives early, but the online algorithm selects another segment with a higher priority.
- C3. Segment s^* arrives after the online algorithm has already selected another segment for execution. Due to the non-preemptive nature of the algorithm, s^* cannot be executed during this interval.

For C1,

$$U(W(s^*); x^*) \leq U(W(s); x)$$

For C2, s is chosen via $U(\cdot) e^{\min(0, \Delta w(s_k^i, t))}$, we must have

$$U(W(s); x) e^{\min(0, \Delta w(s, t))} \geq U(W(s^*); x^*) e^{\min(0, \Delta w(s^*, t))}$$

Rearranging the terms, we obtain:

$$U(W(s^*); x^*) \leq \frac{e^{\min(0, \Delta w(s, t))}}{e^{\min(0, \Delta w(s^*, t))}} U(W(s); x)$$

We further analyze the underlying reasons for this phenomenon. As illustrated in Fig. 17, we present a concrete example. At time t_1 , both $R1-G2$ and $R2-G2$ arrive. Since their utilities are identical at t_1 and the online algorithm lacks foresight regarding the future arrival of $R3-G2$, it selects $R1-G2$ for generation based on the slack time criterion $e^{\min(0, \Delta w(s_k^i, t))}$. Subsequently, $R3-G2$ arrives with a

higher priority, resulting in its generation before $R2-G2$. Under the online strategy, the total utility accumulated is given by:

$$U(s_2^1, t1) + U(s_2^3, t2) + U(s_2^2, t3) = 1 + 2 + 1 - 2 \times \Delta w(s_2^2, t3) = -1$$

In contrast, under the offline optimal strategy, the arrival of $R3-G2$ is known in advance, allowing for a globally optimal decision that prioritizes the generation of $R2-G2$ first. Consequently, under the offline optimal strategy, the total utility obtained is computed as:

$$U(s_2^2, t1) + U(s_2^3, t2) + U(s_2^1, t3) = 1 + 2 + 1 - 1 \times \Delta w(s_2^1, t3) = 2.5$$

The underlying principle is that myopically selecting tasks based on slack time can result in suboptimal scheduling decisions. Our online algorithm cannot anticipate incoming tasks, and the tasks arriving later may cause blockage. If a blockage occurs and the task utility starts to decline, the task degradation rates become crucial. The extent of utility degradation depends on the blocking time (the batch size) and the maximum degradation rate within the task set.

In this case, both slack times $\Delta w(s, t)$ and $\Delta w(s^*, t)$ are negative, neither segment has yet exhausted its buffer. Hence,

$$\frac{e^{\min(0, \Delta w(s, t))}}{e^{\min(0, \Delta w(s^*, t))}} = e^{\Delta w(s, t) - \Delta w(s^*, t)}$$

Suppose the offline-optimal segment s^* is scheduled j^* full intervals later. During these intervals, both utilities continue to decay but remain non-negative, which implies

$$-\Delta w(s^*, t) < j^* \times D_{max} < -\Delta w(s, t) - \frac{\beta_{max}}{\alpha_{max}}$$

Substituting the middle term of the inequality above into the exponent gives

$$e^{\Delta w(s, t) - \Delta w(s^*, t)} < e^{\frac{\beta_{max}}{|\alpha_{max}|}}$$

Therefore, the utility obtained by the offline-optimal schedule for s^* is bounded by

$$U(W(s^*); x^*) \leq e^{\frac{\beta_{max}}{|\alpha_{max}|}} U(W(s); x)$$

By defining $c1 = e^{\frac{\beta_{max}}{|\alpha_{max}|}}$, we obtain the desired local inequality in I_j :

$$\begin{aligned} \sum_{s^* \in O_j} U(W(s^*); x^*) &= U(W(s^*); x^*) \\ &\leq c1 \cdot U(W(s); x) \leq c1 \cdot \sum_{s \in A_j} U(W(s); x) \end{aligned}$$

For **C3**, as shown in Fig. 18, at time t_1 , both $R1-G2$ and $R2-G2$ arrive, and our algorithm select $R1-G2$ for generation. Subsequently, $R3-G2$ arrives with a higher priority, resulting in its generation before $R2-G2$. Under the online strategy, the total utility accumulated is given by:

$$U(s_2^1, t1) + U(s_2^3, t3) + U(s_2^2, t5) = 1 + 2 - 2 \times \Delta w(s_2^3, t3) + 1 = 2$$

In contrast, under the offline optimal strategy, the arrival of $R3-G2$ is known in advance, allowing for a globally optimal decision that prioritizes the generation of $R2-G2$ first. Consequently, under

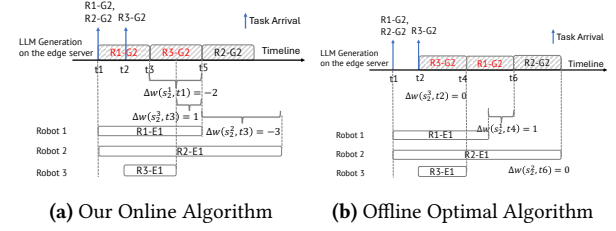


Figure 18: An example for **C3**: At time t_1 , both $R1-G2$ and $R2-G2$ arrive, and our algorithm selects $R1-G2$ for generation. At time t_2 , $R3-G2$ arrives, but its generation is delayed due to the ongoing generation of $R1-G2$. However, the optimal schedule would prioritize generating $R3-G2$ before $R1-G2$. This is because $R3-G2$ has a higher initial utility β , contributing more to the total utility. ($R1: \alpha = -1, \beta = 1, R2: \alpha = -2, \beta = 1, R3: \alpha = -1, \beta = 2, G=2$, batch size=1, R: request, G: generation, E: execution, Δw : slack time.) A key takeaway is that selecting tasks online can lead to suboptimal scheduling, as it cannot foresee future tasks with higher utility. This may result in blocking scenarios where a lower-utility task is prioritized, delaying the generation of a more valuable task. The extent of utility degradation depends on the blocking time (the batch size) and the maximum initial utility (β_{max}) within the task set.

the offline optimal strategy, the total utility obtained is computed as:

$$U(s_2^3, t2) + U(s_2^1, t4) + U(s_2^2, t6) = 2 + 1 - 1 \times \Delta w(s_2^1, t4) + 1 = 3$$

The underlying principle is that, under an online scheduling mechanism, the generation of a new task may be delayed due to blockage by existing running tasks. In contrast, an online algorithm has complete knowledge of all tasks, enabling it to make optimal scheduling decisions and thereby effectively increasing the total time utility. The extent of utility degradation depends on the blocking time (the batch size) and the maximum initial utility within the task set.

We adopt cross-interval charging. Suppose the segment chosen by the offline optimal algorithm is s^* from request r^i , and the online algorithm executes the segment s^* in the interval I_{j^*} , we represent it as $U(W(s); x)$.

In this case, segment s^* is not selected by the online algorithm because another segment has already been chosen for execution; however, since s^* has high potential utility and is selected by the offline optimal, the online algorithm will prioritize it in the next batch, the utility loss is bounded by the reduction due to the generation delay, which is at most $|\alpha_{min}| \cdot D_{max}$. If s^* is not selected in the subsequent batch either, then it falls under **C2**, where the online algorithm consistently selects segments with even higher utility density, leading to potential starvation of s^* . Hence, we have

$$U(W(s^*); x^*) \leq U(W(s); x) + |\alpha_{min}| \cdot D_{max}$$

So we have,

$$\frac{U(W(s^*); x^*)}{U(W(s); x)} \leq \frac{U(W(s); x) + |\alpha_{min}| \cdot D_{max}}{U(W(s); x)} \quad (9)$$

$$\leq 1 + \frac{|\alpha_{min}| \cdot D_{max}}{U_{min}} \quad (10)$$

To establish a strictly bounded competitive ratio in our theoretical analysis, we assume an admission threshold of $U_{min} > 0$. Under this theoretical constraint, any segment s^* successfully generated and counted towards the online algorithm’s total utility strictly satisfies $U(W(s); x) \geq U_{min}$. While our practical implementation relaxes this threshold to drop requests only when their utility becomes negative, this analytical bound guarantees the algorithm’s worst-case stability. Set $c_2 = 1 + \frac{|\alpha_{min}| \cdot D_{max}}{U_{min}}$, we have

$$U(W(s^*); x^*) \leq c_2 \cdot U(W(s); x)$$

For every s^* in $O(M_j) = O_j \cup O_{j+1}$, we can find an s in $A(M_j) = A_j \cup A_{j+1}$ such that

$$\sum_{s^* \in O(M_j)} U(W(s^*); x^*) \leq c_2 \cdot \sum_{s \in A(M_j)} U(W(s); x)$$

Joint Analysis. Combining the conclusion in **Case 1** and **Case 2**, let $c = \max\{c_1, c_2\} = \max\{e^{\frac{\beta_{max}}{|\alpha_{max}|}}, 1 + \frac{|\alpha_{min}| \cdot D_{max}}{U_{min}}\}$. We first merge two intervals into one. Then we have merged intervals $M_0 = I_0 \cup I_1$, $M_2 = I_2 \cup I_3$. For every merged interval M_{2j} , we have:

$$\sum_{s^* \in O(M_{2j})} U(W(s^*); x^*) \leq c \cdot \sum_{s \in A(M_{2j})} U(W(s); x).$$

By summing j from 0 to $\lfloor (J+1)/2 \rfloor - 1$, we obtain:

$$\sum_j \sum_{s^* \in O(M_{2j})} U(W(s^*); x^*) \leq c \cdot \sum_j \sum_{s \in A(M_{2j})} U(W(s); x)$$

Since $\bigcup_j O(M_{2j}) = \bigcup_{j=0}^J O_j$ and $\bigcup_j A(M_{2j}) = \bigcup_{j=0}^J A_j$, we obtain:

$$v(x^*) = \sum_{j=0}^J \sum_{s^* \in O_j} U(W(s^*); x^*) \leq c \cdot \sum_{j=0}^J \sum_{s \in A_j} U(W(s); x) = c \cdot v(x).$$

This completes the competitive analysis of **Theorem 2**. Hence, the competitive ratio of our algorithm depends on the batch size as well as the time-sensitive task parameters, such as α, β .

C Artifact Appendix

C.1 Abstract

This artifact accompanies our paper, “*TimelyLLM: Time-Sensitive LLM Serving System for Physical-I/O Limited Agents*.” It includes the TIMELYLLM source code, a Jupyter notebook for one-click experiment execution, example workloads derived from our LRTrace framework, and configuration files for reproducing the paper’s main evaluation results. The artifact is designed to demonstrate the core mechanisms of TIMELYLLM, including content-aware segmented generation, slack-based priority scheduling, as well as their impact on time-sensitive multi-agent workloads.

Using the provided scripts and notebook, users can reproduce the key experimental results reported in §7. Compared with a vLLM-based baseline, TIMELYLLM improves time utility on representative physical-I/O limited workloads. It also includes a scheduling ablation study and an evaluation on a different robotic system (the FLTRNN robot arm).

C.2 Artifact check-list (meta-information)

- **Algorithm:** Content-aware segmented generation; slack-based priority scheduling; latency-guided continuous batching.
- **Program:** Python implementation built on top of vLLM.
- **Compilation:** none.
- **Transformations:** request segmentation; scheduler-driven priority assignment.
- **Binary:** Docker image for reproduction.
- **Model:** Meta-Llama-3-8B-Instruct.
- **Data set:** representative subsets of LRTrace-derived robotic traces (drone and robot arm); preprofiled scheduling metadata.
- **Run-time environment:** Linux; Docker; Python 3.10; CUDA-compatible GPU environment.
- **Hardware:** one NVIDIA RTX 4090 GPU; 24 GB GPU memory recommended.
- **Run-time state:** generated logs, JSON files, and output figures.
- **Execution:** Jupyter notebook or CLI with preset configurations.
- **Metrics:** time utility; response time; waiting time;
- **Output:** log files; JSON data files; reproduced figures; scheduler logs.
- **Experiments:** performance comparison with vLLM (Exp 7.4.1); scheduling algorithm ablation (Exp 7.4.3); different robotic systems (Exp 7.4.4).
- **How much disk space required (approximately)?:** 15–30 GB.
- **How much time is needed to prepare workflow (approximately)?:** 10–20 minutes.
- **How much time is needed to complete experiments (approximately)?:** ~15 minutes for Exp 7.4.1; ~20 minutes for Exp 7.4.3; ~25 minutes for Exp 7.4.4.
- **Publicly available?:** Yes.
- **Code licenses:** The source code is released under the MIT License.
- **Data licenses:** The dataset is released under the CC BY 4.0 license.
- **Workflow automation framework used?:** Jupyter notebook with preset-based experiment configurations.
- **Archived?:** Will maintain our public codebase on GitHub.

C.3 Description

C.3.1 How to access. The code can be accessed through the following GitHub repository: <https://github.com/Neawhen/TimelyLLM>. The artifact is provided as a source-code repository with a Jupyter notebook (`run_experiments.ipynb`) that automates environment setup, experiment execution, and result visualization. A Dockerfile is also provided for containerized execution.

Recommended cloud setup on Runpod. We tested the artifact on a Runpod Pod with:

- 1× NVIDIA GeForce **RTX 4090**
- the **RunPod PyTorch 2.2.0** Pod template, with SSH and JupyterLab enabled
- **20GB** Container Disk
- **60GB** Volume Disk
- Follow **Option A: Jupyter Notebook (recommended)** using the **Direct environment setup** path in the GitHub README.

We additionally maintain a pre-configured cloud/JupyterLab environment for convenience. The main artifact is fully reproducible using the instructions above and does not require author intervention.

C.3.2 Hardware dependencies. The main artifact is designed as a software-only evaluation path and does not require physical robots. Reproducing the main results requires a CUDA-capable NVIDIA GPU with at least 24 GB of memory. The artifact has been tested on a single NVIDIA RTX 4090.

C.3.3 Software dependencies. Two execution methods are supported: Local: Python 3.10, uv package manager, NVIDIA GPU with CUDA 12.1 compatible driver. Docker: Linux with Docker, NVIDIA Container Toolkit, and a compatible NVIDIA driver.

C.3.4 Datasets. The artifact contains representative subsets of our workload traces derived from LRTrace, including drone and robot arm traces. These subsets are located in the `dataset/` directory (`data_sample_*.json`).

C.3.5 Models. The artifact uses Meta-Llama-3-8B-Instruct as the LLM backbone. The model can be downloaded from Hugging Face using the provided notebook cell or manually.

C.4 Installation

Clone the GitHub repository to your local Linux machine. Follow the README to setup.

C.5 Experiment workflow

The artifact provides three main experiment workflows, all runnable through the Jupyter notebook `run_experiments.ipynb`.

Experiment 7.4.1 — Performance comparison with vLLM. This workflow compares TIMELYLLM with the vLLM baseline on drone (TypeFly) tasksets. Expected runtime: ~15 minutes.

Experiment 7.4.3 — Scheduling algorithm comparison. This workflow compares the TIMELYLLM scheduler against EDF and FCFS. Expected runtime: ~20 minutes.

Experiment 7.4.4 — Different robotic systems. This workflow evaluates TIMELYLLM on FLTRNN (robot arm), extending the evaluation beyond TypeFly (drone). Expected runtime: ~25 minutes.

Result processing and visualization. After each run, logs are processed by `resdata.py`, and the resulting JSON files are saved under `timelyllm/logs/`. Figures are then generated by the scripts in `fig_plot/`. In the notebook, these steps are invoked automatically.

C.6 Evaluation and expected results

The artifact is intended to reproduce the following key results:

- **Main system result (Exp 7.4.1):** TIMELYLLM improves time utility and reduces response time over the vLLM-based baseline on representative multi-agent drone workloads.
- **Scheduling ablation (Exp 7.4.3):** the TIMELYLLM scheduler outperforms both EDF and FCFS, showing the benefit of slack-based priority scheduling.
- **Generality result (Exp 7.4.4):** TIMELYLLM also improves time utility on FLTRNN robot arm workloads, demonstrating generalization beyond TypeFly drone tasks.

Exact values may vary slightly across hardware and runtime environments, but the qualitative trends above should remain consistent.

C.7 Experiment customization

The artifact supports customized experiments beyond the default paper workflows. After cloning the repository, users can follow the `README.md` instructions to set up the environment and run experiments through either Jupyter notebook or CLI. We provide experiment presets for the main results in the paper, and these presets can be customized by overriding parameters such as `-run-mode` and `-model-path`. The modular code structure also allows users to extend the artifact to new workloads, configurations, and evaluation settings.